Homework 2 Conceptuals: Beras

CS 1470/2470: Deep Learning

Due February 23, 2024 at 6:00pm EST

1 Conceptual Questions

1. The update rule for weights in a single layer, multi-class neural network with cross entropy loss is defined as follows:

   Let w_{ij} be the weight associating the ith input feature x_i with the jth class. Let c be the index of the correct class for a given input (the *label*). The loss and its derivatives are then:

   $$L = -\log(P_c), \qquad \frac{\partial L}{\partial w_{ij}} = \begin{cases} (P_j - 1)x_i & j = c \\ P_j x_i & j \neq c \end{cases}$$

   We use these partials with a learning rate α to descend along the gradient:

   $$w_{ij} = w_{ij} - \frac{\partial L}{\partial w_{ij}}\alpha$$

   Derive the above rules from the original definition of cross-entropy loss.

   Hint: Consider both cases of j, and start by expanding out L.

[ANSWER] Cross-Entropy Loss Definition
The cross-entropy loss for a multi-class classification problem is:

$$L = -\sum_{j=1}^{m} y_j \log(P_j)$$

where: - m is the number of classes. - y_j is the binary indicator of the correct class c (1 if the class is correct, 0 otherwise). - P_j is the predicted probability of class j, computed using the softmax function on the logits.
For a given input, since y_j is 1 for the correct class and 0 for all other classes, the loss simplifies to:

$$L = -\log(P_c)$$

Differentiation of Cross-Entropy Loss
To compute the update rule for the weight w_{ij}, we differentiate the loss L with respect to w_{ij}.
Case 1: $j = c$ (Correct Class)
For the correct class, the derivative is:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial P_c} \cdot \frac{\partial P_c}{\partial z_c} \cdot \frac{\partial z_c}{\partial w_{ij}}$$

Given $L = -\log(P_c)$, we have:

$$\frac{\partial L}{\partial P_c} = -\frac{1}{P_c}$$

For the softmax function, $P_j = \frac{e^{z_j}}{\sum_{l=1}^{m} e^{z_l}}$, and thus $\frac{\partial P_c}{\partial z_c}$ simplifies to $P_c(1 - P_c)$.
$\frac{\partial z_c}{\partial w_{ij}}$ is x_i.
Hence, for $j = c$, we get:

$$\frac{\partial L}{\partial w_{ij}} = (P_c - 1)x_i$$

Case 2: $j \neq c$ (Incorrect Class)
For $j \neq c$, the derivation is similar, but the result is different due to the nature of softmax differentiation for non-target classes:

$$\frac{\partial L}{\partial w_{ij}} = P_j x_i$$

[**ANSWER**] The update rule:

$$w_{ij} = w_{ij} - \alpha \frac{\partial L}{\partial w_{ij}}$$

Substituting our derived partial derivatives:
- For the correct class ($j = c$): $w_{ij} = w_{ij} - \alpha(P_c - 1)x_i$ - For the incorrect class ($j \neq c$): $w_{ij} = w_{ij} - \alpha P_j x_i$

2. In classification problems, we assign a likelihood probability to each class and use a loss function that outputs a loss based on this probability. Why is it not an ideal strategy to use MSE loss for classification tasks? Why do most use cross-entropy loss instead? (*2-4 sentences*)

   Hint: Think about how each loss function is shaped, the inputs that they take in, and the range of each function's inputs.

   [**ANSWER**] Considering the shape, inputs, and range of loss functions, using MSE for classification tasks is not ideal because MSE, designed for continuous outputs, does not effectively handle the categorical nature of classification where outputs are probabilities (between 0 and 1). MSE's quadratic nature also means it doesn't penalize misclassifications as effectively as needed in classification problems, leading to slower convergence. Cross-entropy loss, on the other hand, is specifically tailored for probabilities, penalizing incorrect confident predictions more severely and aligning closely with the goal of maximizing the accuracy of categorical outcomes, thus making it more suitable and efficient for classification tasks.

3. The following questions will refer to gradient descent! Please feel free to refer to lecture notes when applicable.

   (a) In lecture, you learned about gradient descent. What is a gradient? How is it different from a partial derivative? How do they relate? (*2-4 sentences*)

   > **[ANSWER]** A gradient is a vector that represents the direction and rate of the steepest ascent of a function at a given point, consisting of partial derivatives of the function with respect to each of its variables. A partial derivative, on the other hand, measures how a function changes as only one variable is varied while keeping the others constant. They relate in that the gradient combines all the partial derivatives of a multivariable function into a single vector, indicating the direction in which the function increases most rapidly. Thus, while a partial derivative gives the rate of change along a specific axis, the gradient provides a comprehensive view of how the function changes in the multidimensional space.

   (b) Consider the formula for updating our weights:

   $$\Delta w = -\alpha \frac{\partial L}{\partial w}$$

   Why do we negate this quantity? What purpose does this serve? (*2-4 sentences*)

   > **[ANSWER]** We negate the quantity $\alpha \frac{\partial L}{\partial w}$ in the formula for updating our weights to ensure that the update moves the weights in the direction of steepest descent, aiming to minimize the loss function L. The negation serves the purpose of moving against the gradient, which points towards the direction of steepest ascent (increase in L). By subtracting this negative update from the current weights, we effectively move towards reducing the loss, improving the model's predictions by adjusting the weights in the direction that decreases L the fastest, according to the current gradient. This process is fundamental in gradient descent optimization, facilitating the iterative improvement of model parameters.

   (c) During gradient descent, we calculate the partial derivative of loss (L) with respect to each weight (w_{ij}). Why must we do this for every weight? Why can't we do this for some weights? (*1-3 sentences*)

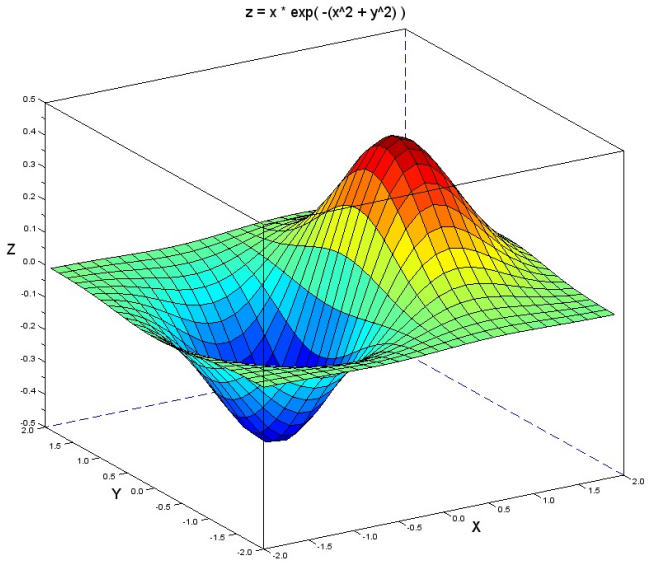
[ANSWER] We calculate the partial derivative of the loss (L) with respect to each weight (w_{ij}) during gradient descent to understand how each weight individually contributes to the overall loss. This comprehensive calculation is necessary because each weight impacts the model's output differently, and adjusting any one weight can influence the loss in a unique way. Omitting the derivative calculation for some weights would ignore their specific impact on the loss, potentially leading to suboptimal adjustments and hindering the model's ability to accurately minimize the loss and converge to the best set of parameters.

(d) In practice, most operations during gradient descent are vectorized. Why do we do this? Why might this make it beneficial to train the model on a GPU? (*1-2 sentences*)

[ANSWER] Vectorization allows for the simultaneous computation of gradients and updates for all weights, significantly speeding up the training process by leveraging the parallel processing capabilities of hardware, especially GPUs. Training on a GPU becomes beneficial because GPUs are designed for high-throughput parallel computations, making them exceptionally efficient at handling the large-scale, simultaneous mathematical operations involved in vectorized gradient descent.

(e) Consider the following plot of a loss function for some neural network:

Figure 1: Loss Manifold Visualization



Where should gradient descent end up on the graph? How many weights does this model have? If our model starts training at $(-2, 2, 0)$,

will the loss function ever reach the absolute minimum? Why? Assume the loss function at this point is *perfectly flat*. (*3-5 sentences*)

[ANSWER] Gradient descent should end up at the bottom of the valley in the graph, which is the point where the loss function reaches its minimum. From the graph, this would be at the origin point $(0, 0)$ where the surface is lowest.
The model depicted in the graph appears to have two weights, corresponding to the x and y axes.
If our model starts training at $(-2, 2, 0)$, the loss function may not reach the absolute minimum due to the starting point being at a flat region of the loss surface, as indicated by the assumption that the loss function is perfectly flat at this point. In a perfectly flat region, the gradient is zero, which means that gradient descent has no direction to follow for improvement, and as a result, the weights would not be updated from their initial values, thus never reaching the absolute minimum.

4. We have previously worked on single-layer linear regression using one linear function $\mathbf{x} \mapsto \mathbf{W_1 x} + \mathbf{b_1}$ mapping from $\mathbb{R}^s$ to $\mathbb{R}^t$. For many real-world scenarios, we actually need multiple layers to model more complex relationships.

   (a) Calculate the result after we stack another linear function $\mathbf{x} \mapsto \mathbf{W_2 x} + \mathbf{b_2}$ mapping from $\mathbb{R}^t$ to $\mathbb{R}^u$ right after the first one.

   [ANSWER] Stacking two linear functions one after the other results in another linear function. The overall transformation from $\mathbb{R}^s$ to $\mathbb{R}^u$ can be calculated as follows:
   1. Apply the first linear transformation: $\mathbf{y} = \mathbf{W_1 x} + \mathbf{b_1}$. 2. Then apply the second linear transformation to the result: $\mathbf{z} = \mathbf{W_2 y} + \mathbf{b_2}$.
   Substituting the expression for $\mathbf{y}$ into the second equation, we get $\mathbf{z} = \mathbf{W_2}(\mathbf{W_1 x} + \mathbf{b_1}) + \mathbf{b_2}$.
   Expanding this out, we find that the result is a new linear transformation of $\mathbf{x}$:
   $\mathbf{z} = (\mathbf{W_2 W_1})\mathbf{x} + (\mathbf{W_2 b_1} + \mathbf{b_2})$.

   (b) What is the shape of $\mathbf{W_1}, \mathbf{b_1}$ and $\mathbf{W_2}, \mathbf{b_2}$? Explain your reasoning.

   [ANSWER] The shapes of $\mathbf{W_1}, \mathbf{b_1}$ and $\mathbf{W_2}, \mathbf{b_2}$ are determined by the dimensions they are mapping from and to. For the first linear function mapping from $\mathbb{R}^s$ to $\mathbb{R}^t$, $\mathbf{W_1}$ must be $t \times s$, and $\mathbf{b_1}$ must be $t \times 1$. For the second mapping from $\mathbb{R}^t$ to $\mathbb{R}^u$, $\mathbf{W_2}$ must be $u \times t$, and $\mathbf{b_2}$ must be $u \times 1$.

   (c) Does the composition of the two linear functions offer an improvement over a single linear function? Explain (*2-4 sentences*)

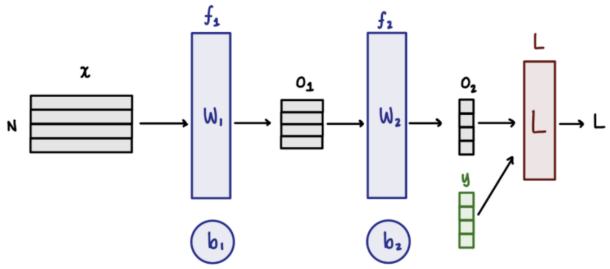
   [ANSWER] The composition of the two linear functions does not inherently offer an improvement over a single linear function in terms of expressive power. This is because their composition is equivalent to a single linear transformation, as shown in the expanded expression for $\mathbf{z}$. Linear functions, whether composed or single, cannot model complex relationships that involve nonlinearity. To capture nonlinearity and therefore increase the model's complexity and capability to learn more complex functions, nonlinear activation functions are needed between the linear layers.

2 Bonus Questions

Note: 2470 Questions are written below!

*The following questions are *bonus* questions and are **not required**. If you do not understand the 'compose' or 'get' gradient functions, TAs will redirect you to these questions first. Completing these questions **will help** you complete the assignment.*

Figure 2: Sample Neural Network



For the following questions, the answers will have to do with the following four functions: *get_input_gradients()*, *get_weight_gradients()*, *compose_input_gradients()*, and *compose_weight_gradients()*. Note that through the Diffable class, each of the layers shown above (f_1, f_2, L) either individually implement or inherit these functions.

1. To initialize backpropagation, which function must you call on which layer? What partial derivative(s) would this return? (Hint: Think back to HW1)

   [ANSWER] get_input_gradients() on the lost layer. The partial derivative of the loss with respect to the predicted value.

2. At layer f_1, what shape are the partials that 'get_input_gradients()' and 'get_weight_gradients() 'return? Imagine that x is a vector of length m and that w_1 is a weight matrix of size (m, r). 'J' will be the correct upstream gradient. (Hint: Think back to HW1)

   [ANSWER] (353,1) and (10,2)

3. At layer f_1, what shape are the partials that 'compose_input_gradients(J)' and 'compose_weight_gradients(J)' return? For this to occur, how must each function resolve the input dimension? What upstream partial is necessary for these functions?

   [ANSWER] I wonder...

4. At layer f_1, how does 'compose_input_gradients(J)', resolve the input dimension difference? More specifically, how does this function create a gradient the same shape as the inputs to this layer (N, m)? Be sure to reference the matrix multiplication at line 200 in 'core.py'. Imagine that w_2 is a weight matrix of size $(r, 1)$.

[ANSWER] I wonder...

3 2470-only Questions

The following are questions that are only required by students enrolled in CSCI2470.

1. In your introductory calculus class, you were likely exposed to the following simple algorithm for finding the minimum of a function: take the derivative of the function, set it to zero, then solve the equation.

   Neural networks are differentiable, so why don't we use this algorithm (instead of gradient descent) to minimize the loss? (*1-4 sentences*)

   [ANSWER] This direct approach of setting the derivative to zero to find minima is not feasible for neural networks due to their complexity and high dimensionality, which often results in a loss surface with numerous local minima, maxima, and saddle points. Solving the equations analytically is typically intractable for neural networks because the loss function is not a simple curve but a high-dimensional landscape. Gradient descent, on the other hand, is an iterative numerical method that uses the gradient to make small, stepwise adjustments to the network's weights, allowing for practical optimization even in complex and high-dimensional spaces where analytical solutions are not possible.

2. Prove that SGD with batch size of 1 gives an *unbiased estimate* of the "true gradient" (the gradient calculated over the entire training set). Assume that the batch is selected uniform-randomly from the full training set.

   Hints:

   (a) Recall that an estimator $\hat{f}$ is an unbiased estimator of a function f if $\mathbb{E}[\hat{f}] = f$.

   (b) Both expectation and differentiation are linear operators.

[ANSWER] Let's denote the loss function for a single data point i as $L_i(\mathbf{w})$, where $\mathbf{w}$ represents the weights of the neural network. The true gradient of the loss function over the entire training set of size N is the average of the gradients of the loss for each individual data point:

$$\nabla L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \nabla L_i(\mathbf{w})$$

Now, let's consider the expectation of the gradient estimate $\hat{\nabla} L(\mathbf{w})$ when a single data point is chosen uniformly at random for the batch:

$$\mathbb{E}[\hat{\nabla} L(\mathbf{w})] = \mathbb{E}[\nabla L_i(\mathbf{w})]$$

Since each data point is equally likely to be chosen (with probability $1/N$), the expectation of the gradient estimate is:

$$\mathbb{E}[\hat{\nabla} L(\mathbf{w})] = \frac{1}{N} \sum_{i=1}^{N} \nabla L_i(\mathbf{w})$$

This is because the expectation of a random variable is the sum of the probabilities of each outcome multiplied by the value of the outcome, and each gradient $\nabla L_i(\mathbf{w})$ is weighted by $1/N$ in the sum.
The linearity of expectation and differentiation implies that the order of the expectation and gradient operators can be interchanged. Therefore, the expected value of the estimate is the same as the true gradient:

$$\mathbb{E}[\hat{\nabla} L(\mathbf{w})] = \nabla L(\mathbf{w})$$

Thus, SGD with a batch size of 1 gives an unbiased estimate of the true gradient because the expected value of the gradient estimate is equal to the actual gradient over the entire training set.

In the assignment, you will see that even a single-layer neural network can classify the digit with pretty good accuracy. Still, there are several ways in which one could improve the accuracy of the model. The paper in the link below suggests and easy way to increase the performance without changing the architecture of our network: training several identical models! Please read the following paper (found here) and answer the following questions:

You may need to login w/ your Brown username to access the paper.

3 What is the committee? Why would it help increase the performance of the model?

[**ANSWER**] The committee is a group of classifiers that are trained on the same data but with different preprocessing techniques. The aim of the committee is to produce a group of classifiers whose errors are not correlated. This is achieved by training identical classifiers on data that are preprocessed in different ways.The committee helps increase the performance of the model because different classifier designs offer complementary information.By combining the predictions of multiple classifiers, the committee can make more accurate predictions and achieve better recognition rates.

4 What is the preprocessing they implement in this paper? How does this preprocessing prevent the models' errors from being strongly correlated?

[**ANSWER**] The preprocessing implemented in this paper involves two main steps: width normalization and deformation.
Width normalization: The authors observed that the aspect ratio of handwritten digits in the MNIST dataset varied significantly. To address this, they normalized the width of the bounding box for each digit to be between 8 and 20 pixels, with a step size of 2 pixels.This resulted in 7 different training sets, each with a different width normalization.Additionally, for the digit "one," they did not perform any width normalization.
Deformation: In addition to width normalization, the authors applied deformations to the digits prior to training. The deformations included elastic distortions, scaling (horizontal and vertical), rotation, and horizontal shearing. The parameters for these deformations were randomly selected within certain ranges. For example, the elastic distortions were characterized by parameters and , which emulated uncontrolled oscillations of hand muscles. The rotation and horizontal shearing were described by a random angle and a ratio tan , respectively. The horizontal and vertical scaling parameters x and y were randomly selected from a range.
The combination of width normalization and deformation helps prevent the errors of the individual models from being strongly correlated. By applying different preprocessing techniques to each individual model, the models are exposed to different variations of the input data. This leads to a diversity of errors among the models, as they learn to recognize the digits in different ways. As a result, when the models are combined in a committee, their errors are not strongly correlated, allowing for improved performance.